

MERN Stack Mini Project

Lab Question Paper — 4 Scenarios | Node.js • Express • MongoDB • React

Trainer: Anish S Nair | MERN Stack Bootcamp — MCA Batch | Duration: 1 Day (Lab)

This paper contains **four project scenarios**. Each team will be assigned **one** scenario. The requirements below are **common to all scenarios**. You are free to design your own project structure, UI layout, and code organization.

Common Requirements (Applicable to All Scenarios)

A. Backend — Node.js + Express + MongoDB

- Create an API to **Add** a record using the **POST** method.
- Create an API to **View All** records using the **POST** method.
- The record must contain **all 13 fields** specified in your scenario's field table.
- Data must be stored in **MongoDB**.
- Test both APIs using Postman / Thunder Client before frontend integration.

B. Frontend — React

- **Add page** — a form with all 13 fields. On submit, it must call the Add API and show a success / error message.
- **View All page** — must call the View API and display all records in a **table view**.
- Both APIs must be **integrated into the React UI** — no hardcoded data.

C. Git Versioning (Mandatory — on BOTH repositories)

Both the backend and frontend repositories must follow this branching strategy:

```
main
  ■■■ development
    ■■■ feature/add
    ■■■ feature/view
```

- Develop the Add functionality in **feature/add** and the View functionality in **feature/view**.
- Merge feature branches into **development**, and finally **development** → **main**.
- Direct commits to main are not allowed.

D. Submission

- GitHub URLs of both repositories (all branches visible).
- Screenshots of both APIs tested in Postman / Thunder Client.
- Screenshots of the Add form and the View table with at least 5 records.

Note: Design decisions — folder structure, styling, validations, and naming — are entirely yours. Copied repositories will be awarded zero.

Scenario 1 — Hackathon Team Management System

Problem Statement

Your college is hosting a 24-hour national-level hackathon. Build a MERN stack application for the organizing committee to **register hackathon teams** and **view all registered teams** with their allotted table/station.

Required Functionalities

- API 1 (POST): **Add Team** — accepts all 13 fields listed below.
- API 2 (POST): **View All Teams** — returns all records.
- React Page 1: **Add Team** form integrated with API 1.
- React Page 2: **View All Teams** table integrated with API 2.

Required Fields (12 Fields + Table / Station Number = 13 Total)

#	Field Name	#	Field Name
1	Team ID	8	Project Title
2	Team Name	9	Problem Statement Track
3	Team Leader Name	10	Technology Stack
4	Leader Email	11	Mentor Name
5	Leader Phone	12	Registration Date
6	College Name	13	Table / Station Number
7	Number of Members		

Follow the **Common Requirements** (Backend, Frontend, Git Versioning, Submission) given on the first page.

Scenario 2 — NSS Volunteer Management System

Problem Statement

The NSS cell of your college needs a MERN stack application to **register NSS volunteers** for camps and activities, and **view all volunteers** along with their allotted unit.

Required Functionalities

- API 1 (POST): **Add Volunteer** — accepts all 13 fields listed below.
- API 2 (POST): **View All Volunteers** — returns all records.
- React Page 1: **Add Volunteer** form integrated with API 1.
- React Page 2: **View All Volunteers** table integrated with API 2.

Required Fields (12 Fields + Unit Number = 13 Total)

#	Field Name	#	Field Name
1	Volunteer ID	8	Department
2	Full Name	9	Year of Study
3	Email	10	Camp Name
4	Phone	11	Hours Completed
5	Date of Birth	12	Address
6	Gender	13	Unit Number
7	Blood Group		

Follow the **Common Requirements** (Backend, Frontend, Git Versioning, Submission) given on the first page.

Scenario 3 — EV Charging Station Management System

Problem Statement

A public EV charging station wants a MERN stack application to **register electric vehicles for charging slots** and **view all bookings** with their allotted charging bay.

Required Functionalities

- API 1 (POST): **Add Vehicle Booking** — accepts all 13 fields listed below.
- API 2 (POST): **View All Bookings** — returns all records.
- React Page 1: **Add Vehicle Booking** form integrated with API 1.
- React Page 2: **View All Bookings** table integrated with API 2.

Required Fields (12 Fields + Charging Bay Number = 13 Total)

#	Field Name	#	Field Name
1	Booking ID	8	Battery Capacity (kWh)
2	Owner Name	9	Connector Type
3	Email	10	Charging Date
4	Phone	11	Time Slot
5	Vehicle Registration Number	12	Estimated Units (kWh)
6	Vehicle Brand	13	Charging Bay Number
7	Vehicle Model		

Follow the **Common Requirements** (Backend, Frontend, Git Versioning, Submission) given on the first page.

Scenario 4 — Pet Boarding Management System

Problem Statement

A pet boarding center wants a MERN stack application to **register pets** checking in for boarding and **view all pets** currently registered with their allotted kennel.

Required Functionalities

- API 1 (POST): **Add Pet** — accepts all 13 fields listed below.
- API 2 (POST): **View All Pets** — returns all records.
- React Page 1: **Add Pet** form integrated with API 1.
- React Page 2: **View All Pets** table integrated with API 2.

Required Fields (12 Fields + Kennel Number = 13 Total)

#	Field Name	#	Field Name
1	Booking ID	8	Owner Name
2	Pet Name	9	Owner Phone
3	Pet Type (Dog/Cat/Bird...)	10	Owner Email
4	Breed	11	Check-in Date
5	Age	12	Check-out Date
6	Weight (kg)	13	Kennel Number
7	Vaccination Status		

Follow the **Common Requirements** (Backend, Frontend, Git Versioning, Submission) given on the first page.

“Compute first, code second — get the API working in Postman before you touch React.”